

kube-prometheus-stack

A Complete, From-Zero DevOps Field Guide

Prometheus · Grafana · Alertmanager · kube-state-metrics · node-exporter · Prometheus Operator

[Core Concepts](#) → [Architecture](#) → [Installation](#) → [PromQL](#) → [Alerting](#) → [Production Practice](#)

Table of Contents

1. Why Monitoring Exists — The Problem Before the Tool
2. Monitoring Landscape — Where kube-prometheus-stack Fits
3. Core Concept: What Is Prometheus, Really?
4. Core Concept: What Is the Prometheus *Operator*?
5. Anatomy of kube-prometheus-stack — Every Component Explained
6. The Full Architecture Diagram & Data Flow
7. Installing kube-prometheus-stack Step by Step (Helm)
8. CRDs You Must Master: ServiceMonitor, PodMonitor, PrometheusRule
9. How a Metric Actually Travels — End-to-End Trace
10. PromQL — The Query Language, From Fundamentals
11. Grafana — Data Source, Dashboards, Panels
12. Alertmanager — Routing, Grouping, Receivers, Silences
13. Requests, Limits & Why HPA Depends on Them
14. Production Checklist — What Real Teams Actually Configure
15. Troubleshooting Playbook
16. Command & Concept Cheat Sheet

1. Why Monitoring Exists

Before touching a single YAML file, internalize the problem monitoring solves. You ship a new build. The deployment pipeline turns green. The pods show Running. Yet users complain: pages load slowly, some feature is broken, checkout fails intermittently. Kubernetes can tell you a pod is *alive* — it cannot tell you the pod is *healthy*, *fast*, or *correctly serving traffic*. Liveness and readiness probes are a shallow signal. They answer "is the process responding to a socket?" not "is CPU exhausted, is memory about to OOM-kill this pod, is one of your 150 microservices silently degrading?"

The core insight: at small scale (4-5 servers, a handful of services) a human can SSH in and check things manually. At real enterprise scale — hundreds of microservices, dozens of namespaces, many nodes — manual inspection is mathematically impossible. You need a system that continuously collects numeric signals (metrics), stores them over time, lets you query them, visualizes them, and pages a human the moment something crosses a dangerous threshold.

That is monitoring's job in one sentence: **collect** → **store** → **query/visualize** → **alert**. Four stages. Every tool in this book maps onto exactly one of those four stages.

Stage	Question it answers	Tool in this stack
Collect	What is happening right now on every pod/node?	Exporters (node-exporter, kube-state-metrics, cAdvisor)
Store	What happened over the last hours/days/weeks?	Prometheus (time-series database, "TSDB")
Query / Visualize	Show me a graph / dashboard of that data	PromQL + Grafana
Alert	Wake a human up when a threshold is crossed	Alertmanager (Slack, Discord, Email, PagerDuty...)

2. Monitoring Landscape

You will hear many names: Prometheus, Grafana, Datadog, New Relic, ELK, Loki, Jaeger. Understand the map first so you never confuse the tool with the category.

Category	What it's for	Open-source example	Paid/SaaS example
Metrics (numbers over time: CPU%, request count)	Trends, capacity, alerting on thresholds	Prometheus + Grafana	Datadog, New Relic
Logs (text events)	"What exactly happened and why"	Loki, ELK/EFK	Datadog Logs, Splunk
Traces (request path across microservices)	"Which of my 12 services made this request slow"	Jaeger, Tempo	Datadog APM

kube-prometheus-stack is a Helm chart bundle that installs the *metrics* pillar (Prometheus + Grafana + Alertmanager) pre-wired together with Kubernetes-native automation, on top of Kubernetes itself. It is the de-facto open-source default because it is free, battle-tested, and vendor-neutral — paid tools like Datadog are typically adopted later once a company has budget and wants less operational overhead.

3. What Is Prometheus, Really?

3.1 The pull model (the single most important idea)

Most people's first assumption is wrong: they think apps *push* metrics to Prometheus. They don't. Prometheus **pulls** (scrapes) metrics from applications on a schedule (default: every 30 seconds). Every application/exporter exposes an HTTP endpoint, conventionally `/metrics`, that returns plain text numbers. Prometheus repeatedly visits that URL and stores whatever it finds, with a timestamp.



Fig 3.1 — Prometheus initiates the connection (pull), not the app pushing to Prometheus.

Why pull, not push? Pull lets Prometheus itself detect when a target has died (a scrape that fails/timeouts is itself a signal). It also centralizes scrape configuration in one place instead of configuring every app to know where to push metrics.

3.2 What a metric actually looks like

Every exporter's `/metrics` endpoint returns plain text like this — this is the raw format Prometheus scrapes:

```
# HELP node_cpu_seconds_total Seconds the CPU spent in each mode
# TYPE node_cpu_seconds_total counter
node_cpu_seconds_total{cpu="0",mode="idle"} 45234.66
node_cpu_seconds_total{cpu="0",mode="user"} 1523.12
# HELP kube_pod_status_phase Pod phase (Running/Pending/Failed)
# TYPE kube_pod_status_phase gauge
kube_pod_status_phase{namespace="default",pod="nginx-abc",phase="Running"} 1
```

Break this down — three parts of every metric:

1. **Metric name** — e.g. `node_cpu_seconds_total`. Convention: `_total` suffix means it's a counter (only ever increases).
2. **Labels** — the key=value pairs in `{ }`. These are what let you filter/group later in PromQL, e.g. "give me only `namespace="prod"`".
3. **Value** — the number itself, always paired with a timestamp when Prometheus stores it.

3.3 The four metric types

Type	Behavior	Real example
Counter	Only ever goes up (resets to 0 on restart)	Total HTTP requests served, total CPU seconds used
Gauge	Goes up and down freely	Current memory usage, number of running pods
Histogram	Buckets of observations (e.g. request latency distribution)	http_request_duration_seconds_bucket
Summary	Like histogram but calculates quantiles client-side	rarely used directly; histogram is preferred

3.4 Prometheus's internal storage (TSDB)

Prometheus is not a general database — it's a purpose-built **Time Series Database**. Each unique combination of metric name + label set is one "time series", stored as a compressed sequence of (timestamp, value) pairs on local disk. By default Prometheus retains ~15 days of data (configurable). This local-disk design is why, at very large scale, teams add **remote_write** to ship data into a long-term store like Thanos, Mimir, or VictoriaMetrics — something to know exists, even if you don't set it up on day one.

4. What Is the Prometheus Operator?

This is the part most tutorials skip and where beginners get lost. Plain Prometheus (without the Operator) is configured through a single static `prometheus.yml` file listing every scrape target by hand — painful to maintain across a fast-changing Kubernetes cluster where pods appear and disappear constantly.

The **Prometheus Operator** solves this the Kubernetes-native way: it introduces new Kubernetes **Custom Resource Definitions (CRDs)** — new "kinds" of YAML objects, just like Deployment or Service are built-in kinds — that let you declare monitoring configuration as ordinary Kubernetes resources. The Operator is a controller that watches those CRDs and automatically regenerates and reloads Prometheus's real config file behind the scenes.

CRD (Custom Resource)	Purpose
<code>Prometheus</code>	Declares "run a Prometheus server with these settings" (replicas, retention, resources)
<code>ServiceMonitor</code>	"Scrape every pod behind this Service, on this port/path"
<code>PodMonitor</code>	Same idea, but targets pods directly (no Service needed)
<code>PrometheusRule</code>	Declares alerting rules and recording rules as YAML
<code>Alertmanager</code>	Declares "run an Alertmanager cluster with these settings"
<code>AlertmanagerConfig</code>	Declares routing/receivers (Slack, email...) as YAML per-team

Mental model: You never hand-edit Prometheus's internal config file. You write small Kubernetes YAML objects (`ServiceMonitor`, `PrometheusRule`...). The Operator is a background robot that watches for those objects and does the file-editing + reload for you. This is exactly the same "declare desired state, let a controller reconcile it" pattern as every other Kubernetes object — Deployments, ArgoCD Applications, everything.

5. Anatomy of kube-prometheus-stack

`kube-prometheus-stack` is a single Helm chart (from the `prometheus-community` chart repository) that installs **all** of the following together, pre-wired to talk to each other:

Component	Kubernetes object type	Job
Prometheus Operator	Deployment	Watches CRDs, manages the whole stack's config
Prometheus server	StatefulSet (via CRD)	Scrapes targets, stores TSDB, evaluates alert rules, runs PromQL queries
Alertmanager	StatefulSet (via CRD)	Receives firing alerts from Prometheus, groups/dedupes/routes/sends notifications
Grafana	Deployment	Reads from Prometheus, renders dashboards, hosts its own alerting UI too
kube-state-metrics (KSM)	Deployment	Converts Kubernetes API object <i>state</i> (Deployments, Pods, PVCs...) into Prometheus metrics
node-exporter	DaemonSet (1 pod per node, always)	Exposes OS/hardware-level metrics: CPU, memory, disk, network per node
Metrics Server (often separate)	Deployment	Powers <code>kubectl top</code> and HPA — NOT part of long-term storage/dashboards

Common confusion: *kube-state-metrics* vs *node-exporter* vs *Metrics Server* — these three sound similar and are the #1 source of interview confusion.

- **node-exporter** = hardware/OS-level facts (CPU%, disk IO, network bytes) — runs as DaemonSet, one per node, talks to the Linux kernel.
- **kube-state-metrics** = Kubernetes *object* facts (how many replicas desired vs available, pod phase, deployment status) — reads the Kubernetes API, not the kernel.
- **Metrics Server** = a lightweight, short-retention metrics pipeline used ONLY by `kubectl top` and the HPA controller. It is not queried by Grafana and is usually installed separately from kube-prometheus-stack.

6. The Full Architecture Diagram & Data Flow

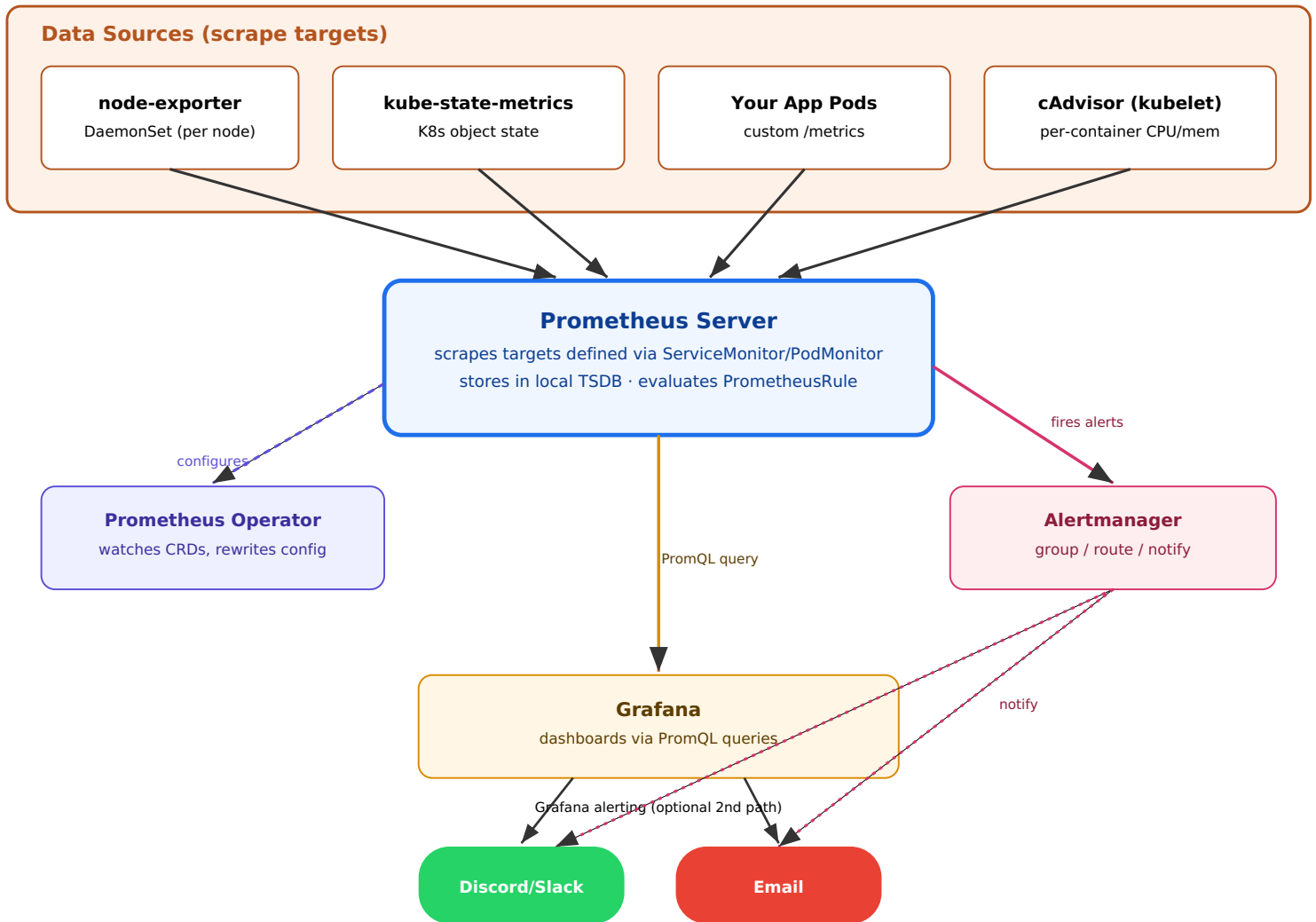


Fig 6.1 — Full data flow: exporters are scraped by Prometheus (configured by the Operator via CRDs); Grafana reads from Prometheus for dashboards; Prometheus itself evaluates alert rules and pushes firing alerts to Alertmanager, which routes them to Slack/Discord/Email.

6.1 Read this diagram like a story

1. You never touch Prometheus's config file directly. You write a small `ServiceMonitor` YAML → the **Operator** notices it → regenerates Prometheus's real scrape config → reloads Prometheus without downtime.
2. **Prometheus** then starts pulling `/metrics` from node-exporter, kube-state-metrics, and your own apps every scrape interval (default 30s), storing everything in its local TSDB.
3. **PrometheusRule** objects define conditions like "CPU > 80% for 5 minutes" — Prometheus itself evaluates these continuously against the stored data.
4. When a rule's condition becomes true, Prometheus marks the alert `firing` and pushes it to **Alertmanager** — a completely separate component whose only job is routing/deduping/notifying, not detecting.
5. **Alertmanager** groups similar alerts, silences known issues, and finally calls a receiver webhook — Slack, Discord, Email, PagerDuty.
6. Independently, **Grafana** is just a read-only visualizer: it sends PromQL queries to Prometheus whenever you open a dashboard, and can optionally run its own separate alerting engine too (a second, UI-friendly alerting path many teams use instead of/alongside PrometheusRule).

7. Installing kube-prometheus-stack Step by Step

7.1 Prerequisites

- A running Kubernetes cluster (EKS, GKE, AKS, kind, minikube — anything) with `kubectl` configured
- Helm 3 installed
- Cluster-admin permissions (the chart installs CRDs, which is a cluster-scoped operation)

7.2 Add the Helm repo

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update
```

Always run `helm repo update` after adding a repo — it refreshes the local index so you install the latest chart version, not a stale cached one.

7.3 Create a dedicated namespace

```
kubectl create namespace monitoring
```

Best practice: keep monitoring in its own namespace, isolated from application workloads — makes RBAC, resource quotas, and cleanup simpler.

7.4 Install the chart

```
helm install monitoring prometheus-community/kube-prometheus-stack \
  --namespace monitoring \
  --create-namespace
```

Breakdown: `monitoring` (first arg) = the Helm **release name** you choose. `prometheus-community/kube-prometheus-stack` = which chart, from which repo. Every resource this chart creates will be prefixed with your release name (e.g. `monitoring-grafana`, `monitoring-kube-prometheus-prometheus`).

7.5 Verify everything came up

```
kubectl get all -n monitoring
kubectl get pods -n monitoring
```

Expect to see: an Operator pod, a Prometheus pod (StatefulSet, name ending `-prometheus-0`), an Alertmanager pod, a Grafana pod, a kube-state-metrics pod, and one node-exporter pod *per node* (DaemonSet).

7.6 Access Grafana

By default the Grafana Service is `ClusterIP` (internal only). For local access, port-forward:

```
kubectl port-forward svc/monitoring-grafana 3000:80 -n monitoring
```

Then open `http://localhost:3000`. Get the admin credentials from the Secret it auto-generated:

```
kubectl get secret monitoring-grafana -n monitoring \
  -o jsonpath="{.data.admin-user}" | base64 --decode; echo
kubectl get secret monitoring-grafana -n monitoring \
  -o jsonpath="{.data.admin-password}" | base64 --decode; echo
```

Why is it in a Secret and base64-encoded? Kubernetes Secrets store arbitrary values as base64 (not encryption — just an encoding for binary-safe storage). You must decode with `base64 --decode` to read the plaintext credential.

7.7 Customize the install with values.yaml (production way)

Running the chart with zero overrides is fine for learning, but real teams always pass a custom `values.yaml` to control retention, resource limits, storage class, and ingress:

```
helm install monitoring prometheus-community/kube-prometheus-stack \
```

```
--namespace monitoring --create-namespace \  
-f values.yaml
```

```
# values.yaml (trimmed example)  
prometheus:  
  prometheusSpec:  
    retention: 15d  
    resources:  
      requests: { cpu: 250m, memory: 512Mi }  
      limits:   { cpu: 1,    memory: 2Gi }  
    storageSpec:  
      volumeClaimTemplate:  
        spec:  
          accessModes: ["ReadWriteOnce"]  
          resources:  
            requests: { storage: 50Gi }  
  
grafana:  
  adminPassword: "ChangeMe123!"  
  persistence:  
    enabled: true  
    size: 10Gi  
  
alertmanager:  
  alertmanagerSpec:  
    resources:  
      requests: { cpu: 50m, memory: 64Mi }  
      limits:   { cpu: 200m, memory: 128Mi }
```

7.8 Upgrading and uninstalling

```
# Apply a values.yaml change to an existing release  
helm upgrade monitoring prometheus-community/kube-prometheus-stack \  
-n monitoring -f values.yaml  
  
# Roll back to previous release if something breaks  
helm rollback monitoring 1 -n monitoring  
  
# Completely remove the stack  
helm uninstall monitoring -n monitoring  
kubectl delete crd -l app.kubernetes.io/part-of=kube-prometheus-stack
```

Helm does NOT delete CRDs on uninstall by default (a safety measure so you don't accidentally wipe all your ServiceMonitors/PrometheusRules cluster-wide). Delete them explicitly only when you are sure nothing else depends on them.

8. CRDs You Must Master

8.1 ServiceMonitor — scrape via a Service

Tells Prometheus: "find the Service matching these labels, and scrape every pod behind it."

```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: my-app-monitor
  namespace: monitoring
  labels:
    release: monitoring      # MUST match Prometheus's serviceMonitorSelector
spec:
  selector:
    matchLabels:
      app: my-app            # matches the Service's labels
  namespaceSelector:
    matchNames: ["default"]
  endpoints:
    - port: metrics          # matches the Service's named port
      path: /metrics
      interval: 30s
```

#1 beginner trap: ServiceMonitor silently does nothing if its `labels.release` doesn't match what your Prometheus CRD's `serviceMonitorSelector` expects (kube-prometheus-stack defaults this to the Helm release name). Always check: `kubectl get prometheus monitoring-kube-prometheus-prometheus -n monitoring -o yaml | grep -A5 serviceMonitorSelector`

8.2 PodMonitor — scrape pods directly (no Service required)

```
apiVersion: monitoring.coreos.com/v1
kind: PodMonitor
metadata:
  name: my-app-podmonitor
  namespace: monitoring
  labels: { release: monitoring }
spec:
  selector:
    matchLabels: { app: my-app }
  podMetricsEndpoints:
    - port: metrics
      path: /metrics
```

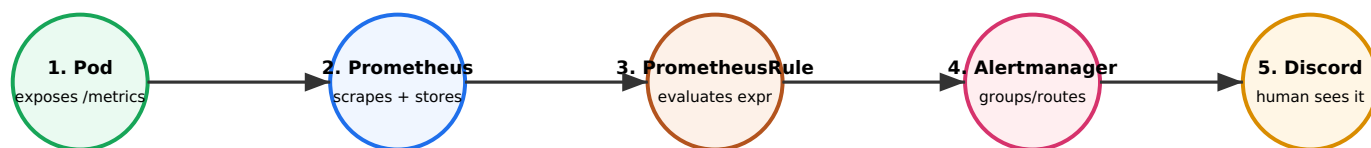
Use PodMonitor when pods don't sit behind a stable Service (e.g. batch Jobs, some StatefulSet patterns) or you want per-pod granularity without going through a Service's load-balancing layer.

8.3 PrometheusRule — alerting & recording rules as YAML

```
apiVersion: monitoring.coreos.com/v1
kind: PrometheusRule
metadata:
  name: high-cpu-alerts
  namespace: monitoring
  labels: { release: monitoring }
spec:
  groups:
    - name: node.rules
      rules:
        - alert: HighNodeCPU
          expr: 100 - (avg by(instance) (rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100) > 80
          for: 5m
          labels:
            severity: warning
          annotations:
            summary: "Node {{ $labels.instance }} CPU above 80%"
            description: "CPU has been above 80% for 5 minutes."
```

Fields explained: `expr` is a PromQL expression evaluated every scrape interval. `for: 5m` means the condition must stay true continuously for 5 minutes before it fires (this prevents noisy one-off spikes from paging someone). `labels.severity` is used later by Alertmanager to route the alert to the right receiver. `annotations` are the human-readable text shown in the notification — templated with Go template syntax like `{{ $labels.instance }}`.

9. How a Metric Actually Travels — End-to-End



Timeline example:

T+0s pod CPU spikes to 95% → T+30s Prometheus scrapes and records the value in TSDB

T+5m PrometheusRule condition "CPU>80% for 5m" becomes true → alert state: pending → firing

T+5m5s Alertmanager receives it, groups with similar alerts, sends to configured receiver (Discord/Slack)

Fig 9.1 — The full lifecycle of one metric, from pod to human notification.

10. PromQL — From Fundamentals

10.1 Instant vector vs range vector

Query	Returns
<code>node_cpu_seconds_total</code>	Instant vector: the latest single value for every matching series right now
<code>node_cpu_seconds_total[5m]</code>	Range vector: every raw sample recorded in the last 5 minutes

10.2 Filtering with labels

```
kube_pod_status_phase{namespace="prod", phase="Running"}
```

This is exactly like a WHERE clause in SQL — filters the time series down to only ones matching the given labels.

10.3 rate() — the most important function you will ever use

Counters (like `node_cpu_seconds_total`) only go up. A raw counter value is useless on its own — you want the *rate of change*. `rate()` computes per-second average increase over a time window:

```
rate(node_cpu_seconds_total{mode="idle"}[5m])
```

This gives "how fast idle-CPU-seconds accumulated, averaged over the last 5 minutes" — from there you derive CPU utilization percentage:

```
100 - (avg by(instance) (rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100)
```

10.4 Aggregation operators

Operator	What it does
<code>sum()</code>	Add values across matching series
<code>avg()</code>	Average across matching series
<code>max()</code> / <code>min()</code>	Highest / lowest value across series
<code>count()</code>	How many series match (e.g. count running pods)
<code>by(label)</code>	Group the aggregation by a specific label, like SQL GROUP BY

10.5 A handful of production-real queries

```
# Pods restarting (crash-loop signal) in the last hour
increase(kube_pod_container_status_restarts_total[1h]) > 0

# Container memory usage vs its limit, as a percentage
sum(container_memory_working_set_bytes{container!=""}) by (pod)
/ sum(kube_pod_container_resource_limits{resource="memory"}) by (pod) * 100

# Number of pods NOT in Running phase, per namespace
count(kube_pod_status_phase{phase!="Running"}) by (namespace)

# 95th percentile HTTP request latency (using a histogram)
histogram_quantile(0.95, rate(http_request_duration_seconds_bucket[5m]))
```

11. Grafana — Data Source, Dashboards, Panels

11.1 How Grafana connects to Prometheus

kube-prometheus-stack automatically pre-configures a Grafana **data source** pointing at the in-cluster Prometheus Service — you don't need to configure this by hand on first install. Verify under *Configuration → Data Sources*.

11.2 Dashboards ship pre-built

The chart auto-imports dozens of community dashboards (node overview, pod overview, Kubernetes cluster overview) via ConfigMaps that Grafana's sidecar container watches and loads automatically. You rarely need to build dashboards from zero — instead, import a known-good community dashboard by ID from grafana.com/grafana/dashboards, then customize.

11.3 Import flow

1. Grafana UI → **Dashboards → New → Import**
2. Paste the numeric dashboard ID (found on grafana.com) or upload exported JSON
3. Select your Prometheus data source when prompted
4. Click Import

11.4 Why Grafana alerting requires a "time series" panel type

Grafana can only attach an alert rule to a panel visualizing data as a continuous graph (time series), not a static table — an alert condition is fundamentally "did this numeric series cross a threshold," which only makes sense against a graph type panel.

12. Alertmanager — Routing, Grouping, Receivers

12.1 The job split: detection vs notification

Prometheus **detects** ("this condition is true"). Alertmanager **decides what to do about it** (who to notify, how often, whether to suppress duplicates). Keeping these separate means you can change notification behavior without touching detection logic, and vice versa.

12.2 Core concepts

Concept	Meaning
Route	A tree of matching rules deciding which receiver handles which alert (e.g. severity=critical → PagerDuty, severity=warning → Slack)
Receiver	The actual destination integration: Slack webhook, email SMTP, Discord webhook, PagerDuty key, generic webhook
Grouping	Batches multiple related alerts (e.g. all pods down in one namespace) into a single notification instead of spamming one message per alert
Silence	A manually-created, time-boxed rule to mute matching alerts (e.g. during planned maintenance)
Inhibition	Suppress lower-severity alerts when a related higher-severity alert is already firing (e.g. don't alert "high latency" if "service down" is already firing for the same service)

12.3 Example AlertmanagerConfig (Discord/Slack webhook)

```
apiVersion: monitoring.coreos.com/v1alpha1
kind: AlertmanagerConfig
metadata:
  name: team-alerts
  namespace: monitoring
  labels: { release: monitoring }
spec:
  route:
    groupBy: ["alertname", "namespace"]
    groupWait: 30s
    groupInterval: 5m
    repeatInterval: 4h
    receiver: discord-webhook
  receivers:
    - name: discord-webhook
      webhookConfigs:
        - url: https://discord.com/api/webhooks/XXXX/YYYY/slack
```

Timing fields explained: `groupWait` = how long to wait after the first alert in a new group before sending the notification (lets a batch of related alerts arrive together). `groupInterval` = minimum wait before sending a notification about new alerts added to an already-notified group. `repeatInterval` = how long to wait before re-sending the same still-firing alert (prevents endless spam).

13. Requests, Limits & HPA

This connects directly to why kube-prometheus-stack matters for autoscaling. HPA (Horizontal Pod Autoscaler) checks CPU/memory utilization *relative to the container's requested resources*, not its limit.

requests

guaranteed baseline
used by scheduler to
place the pod on a node

**HPA % is measured
against THIS number**

limits

hard ceiling
container throttled (CPU) or
OOM-killed (memory) beyond it

Node capacity

no limit set on a pod?
it can consume the whole
node and crash other pods

= always set limits

Fig 13.1 — Why every production Deployment sets both requests AND limits.

```
resources:
  requests:
    cpu: 250m
    memory: 256Mi
  limits:
    cpu: 500m
    memory: 512Mi
```

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: my-app-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: my-app
  minReplicas: 1
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 50
```

Note this reads from the separate **Metrics Server**, not from Prometheus — HPA is a control-loop decision that needs very fresh, low-latency numbers, which Metrics Server's short-lived in-memory pipeline is built for. Prometheus/Grafana is for humans looking at trends; Metrics Server is for the HPA controller making fast automated decisions.

14. Production Checklist

What experienced teams actually configure before going to production — not just "helm install" and walk away:

Area	Practice	Why
Resources	Set requests/limits on Prometheus, Grafana, Alertmanager pods themselves	Prometheus with no limit can consume a whole node under high cardinality
Retention	Tune <code>retention</code> (time) and <code>retentionSize</code> (disk) explicitly	Default retention can silently fill the PVC and crash the pod
Storage	Use a PVC with a real StorageClass, not emptyDir	emptyDir data is lost on pod reschedule
High Availability	Run 2+ Prometheus replicas, 3 Alertmanager replicas	Single replica = monitoring blind spot if that one node dies
Long-term storage	Add Thanos or Mimir with remote_write for >15 day history	Local Prometheus disk isn't meant for years of data
Cardinality control	Avoid unbounded label values (user IDs, request IDs as labels)	Each unique label combo is a new time series — explodes memory/disk
Security	Put Grafana behind Ingress + TLS + SSO, never expose via NodePort publicly	Default admin/password is a common breach vector
Alert hygiene	Use <code>for:</code> durations and inhibition rules	Prevents alert fatigue from flapping/noisy signals
Cost awareness	Watch log/metric volume when integrating with paid tools (Datadog etc.)	Aggressive load-testing traffic can spike SaaS billing overnight

15. Troubleshooting Playbook

Symptom	Likely cause	Check
Target missing in Prometheus → Status → Targets	ServiceMonitor label mismatch with Prometheus's selector	<code>kubectl get prometheus -n monitoring -o yaml grep Selector</code>
HPA shows "unable to fetch metrics"	Metrics Server not installed / not reachable	<code>kubectl get apiservice v1beta1.metrics.k8s.io</code>
Alert never fires despite condition being true	Wrong AlertmanagerConfig label / route matcher mismatch	<code>kubectl logs -n monitoring alertmanager-monitoring-kube-prometheus-alertmanager-0</code>
Grafana panel empty	Wrong data source selected, or PromQL syntax error	Open panel → Query Inspector → check raw response
Prometheus pod OOMKilled / CrashLoop	High cardinality metrics exhausting memory, or retention too high for the PVC size	<code>kubectl top pod -n monitoring</code> , check <code>tsdb</code> size on disk
node-exporter missing on some nodes	DaemonSet tolerations don't match node taints	<code>kubectl get ds -n monitoring -o wide</code>

16. Command & Concept Cheat Sheet

```
# Helm lifecycle
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update
helm install monitoring prometheus-community/kube-prometheus-stack -n monitoring --create-namespace
helm upgrade monitoring prometheus-community/kube-prometheus-stack -n monitoring -f values.yaml
helm uninstall monitoring -n monitoring

# Access UIs locally
kubectl port-forward svc/monitoring-grafana 3000:80 -n monitoring
kubectl port-forward svc/monitoring-kube-prometheus-prometheus 9090:9090 -n monitoring
kubectl port-forward svc/monitoring-kube-prometheus-alertmanager 9093:9093 -n monitoring

# Inspect
kubectl get pods -n monitoring
kubectl get servicemonitor -A
kubectl get prometheusrule -A
kubectl describe prometheus monitoring-kube-prometheus-prometheus -n monitoring
```

Term	One-line definition
Scrape	Prometheus pulling /metrics from a target over HTTP
Exporter	Any process exposing metrics in Prometheus text format
TSDB	Prometheus's built-in time-series storage engine
Operator	Controller that turns CRDs into real Prometheus config, automatically
ServiceMonitor / PodMonitor	CRD declaring what to scrape
PrometheusRule	CRD declaring alerting/recording rules
Alertmanager	Separate component: routes/dedupes/notifies on firing alerts
PromQL	Prometheus's query language for reading time series data
Cardinality	Number of unique label-value combinations for a metric — the #1 scaling risk
Remote write	Streaming Prometheus data out to a long-term store (Thanos/Mimir/VictoriaMetrics)

How to actually master this: install kube-prometheus-stack on a free-tier cluster (kind/minikube is enough), deliberately break something (kill a pod, stress CPU), watch it appear in Prometheus → Grafana → fire an alert → land in your own Discord channel. Reading this book gives you the map; running the loop yourself end-to-end, more than once, is what makes it permanent knowledge.